
ESE 564 Final Project - Sweeping Robot in MuJoCo

Dylan Li

Department of Computer Engineering
Stony Brook University
Stony Brook, NY 11790
dylan.li@stonybrook.edu

Jacob Helzner

Department of Computer Engineering
Stony Brook University
Stony Brook, NY 11790
jacob.helzner@stonybrook.edu

Maanav Anand Kumar

Department of Computer Science
Stony Brook University
Stony Brook, NY 11790
maanav.anandkumar@stonybrook.edu

Abstract

This report describes our implementation of a robot that grabs a broom and attempts to use the broom to sweep objects on the surface to a goal region. An overhead camera is used for perception. YOLOv8-obb is used to detect the objects and the broom so that the robot is understand the object and broom poses. Reinforcement learning is used in training to teach the robot to sweep, with limited success - possible sources of error are discussed. Finally, we discuss our current model based sweeping sequence, illustrating the enduring value of classical methods to solve robot problems in predictable environments.

1 Introduction

Engineering multi-step tasks remains a fundamental challenge in robotics research. While significant progress has been made in object grasping and pick-and-place tasks, tool-use behaviors such as wielding a broom to sweep objects toward a target require tightly integrating perception, grasping, and motion planning into a coherent pipeline. This work presents an end-to-end manipulation system in which a simulated Franka Panda arm perceives a cluttered tabletop, grasps a broom, and executes a sweeping sequence to consolidate objects toward a designated zone.

A central design choice in our pipeline is the clean separation between grasping and sweeping. Rather than learning a pixel-to-torque policy from raw pixels, we decompose the problem into a grasping module and a sweeping module. Given that grasping a known tool from a known pose is more reliably solved analytically, we implement the grasping module using Jacobian damped least-squares inverse kinematics. For the sweeping module, we first explored a learned policy trained via Deep Deterministic Policy Gradient (DDPG), motivated by the observation that contact-rich sweeping dynamics are difficult to model explicitly and may benefit from reinforcement learning over a continuous action space. However, as we discuss later, the sweeping environment presents unique opportunities for reward exploitation that made policy learning difficult. This motivated a pivot to a model-based multi-step inverse kinematics solution, which we found to be more reliable in practice. In both cases, the handoff between modules is a single joint configuration vector, keeping the interface lightweight and modular.

Perception is handled by a YOLOv8 oriented bounding box (OBB) detector trained entirely on synthetic data rendered in MuJoCo, avoiding time-consuming manual annotation. Overhead RGB-D imagery provides both object detection and 3D localization, supplying the sweeping module with

object poses as structured observations. Together, these components form a reproducible simulation pipeline that requires no manual intervention at runtime.

2 Method

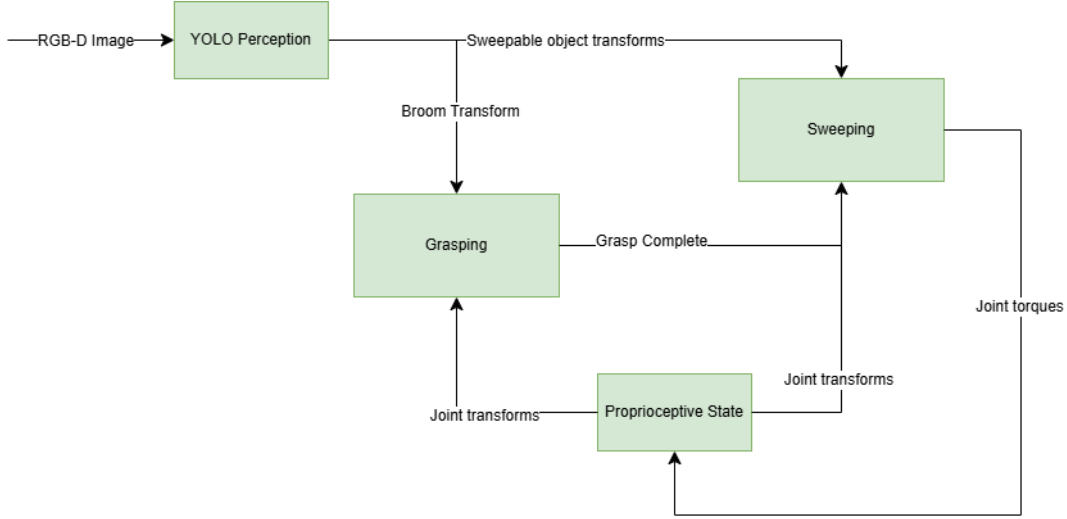


Figure 1: General Pipeline of the Robot Sweeping Model

Our pipeline has three main parts. The first is the YOLOv8-obb perception module, which accepts an RGB-D image and outputs transforms for the broom and all other detected objects in the scene. We finetuned YOLO on synthetic data of the broom, arm and objects by randomizing the positions of each object and calculating their bounding boxes in camera screen space. Only the broom was labelled, in hopes that we would generalize well over the sweepable objects. To actually get the transforms, we did a deprojection using the screen points of the bounding boxes, depth, and camera parameters like the focal length. The next part is the grasping module. This uses the perception module to get the broom transform, and then uses IK to move the arm to a grasping configuration around the broom handle. After this, we tried enabling the RL-trained sweeping policy, which is a neural network.

2.1 RL - Implementation

We used Deep Deterministic Policy Gradient (DDPG) to train our policy. This is a fundamental actor-critic algorithm that works with continuous action spaces. As we discussed in our course, an actor network accepts states and returns joint torques, while a critic network predicts the Q value for that state-action pair. DDPG also uses a replay buffer to help de-correlate consecutive samples.

Actor Network: Accepts the proprioceptive state of the Franka Panda arm, as well as up to 6 detected object transforms. If there are less than 6 detected objects, we zero pad the input. This neural network has 2 hidden layers with 256 neurons each, and the final layer uses a tanh activation which is then scaled to the panda arm torque limits. Additionally, some Gaussian noise is added to training rollouts to the result to encourage exploration.

Critic Network: Accepts the same state as the actor network, alongside the torque outputs. This neural network has the same hyperparameters and architecture as the actor, and outputs the estimated Q value.

Just as was discussed during the lecture, we estimate the true target Q-value obtained from Bellman recursion by taking the actual reward and adding it to a discount factor multiplied by the estimated reward. To help stability, DDPG uses slowly updating *target networks*: specifically, a Target Critic evaluating the next state alongside the action proposed by a Target Actor. These target networks are treated as constants in backpropagation, giving the Critic network a value to optimize during gradient descent that doesn't rapidly fluctuate.

2.2 RL - Reward

We define the reward as follows:

$$R = 25.0 \cdot r_{\text{progress}} + 2.0 \cdot r_{\text{broom}}$$

The progress term r_{progress} is the sum of decreases in the XY distance of each sweepable object to the goal from the last timestep. The broom-head proximity term r_{broom} is the negative XY distance between the broom head and the centroid of all tracked objects, encouraging the policy to keep the broom near the working area. To allow the robot to maneuver around the edges of the object cluster, we scaled down the penalty 0.3x if the broom head is within 0.12 m of any object. Finally, the episode terminates with a penalty of -500 if the broom body drops below $z = 0.03$. This discourages the broom being dropped.

2.3 RL – Possible Sources of Bad Performance

Though the reward eventually stabilized, we never got the policy to actually sweep. Most of our tests, we noticed the arm would settle in a position and stop sweeping, even after correcting for things like passive reward gain. A few of the possible reasons are described below.

Broom proximity relaxation: The policy could achieve a stable, low-penalty state by "parking" the broom head within 0.12m of a single object. This behavior is arguably easy to discover (no movement needed) and dis-incentivizes the robot to move.

Not enough exploration for sustained motions: A real sweeping motion needs directionally correlated torques over many timesteps. This isn't exactly necessary to get the objects in the goal region - in another setup, the arm could feasibly learn to poke objects with the back end of the broom like a pool player - but it's certainly a natural and efficient way. It's possible this type of deliberate motion was not represented well enough in the replay buffer during early exploration, especially since we used a flat gaussian noise with standard deviation of 0.1.

If we were unlimited by the time constraints of the project, our next steps would be the following. First, we could dis-incentivize "freeze" behavior by applying an increasing penalty when broom head position deltas remain low. Since a sweeping behavior doesn't require prolonged stops, this should be appropriate. Second, we would decay the noise level across episodes to encourage more exploration early. This would allow us to get more useful information in the replay buffer before the policy starts to settle.

3 Model Based Solution - Method

Due to the failure of reinforcement learning and the time constraints, we decided to switch to a model based approach. Specifically, inverse kinematics (IK) is used to move the robot with the broom to just behind a detected object, relative to the goal point. This is calculated by going a little further on the line from the goal point to the detected object. Then, once the robot is in position with the broom behind the object, IK is used again to move the robot arm to the goal region to sweep the objects to the goal. The general idea is to create a path for the robot arm to move the broom to the object and then to the goal region using just IK on transforms.

An important feature of our transform-based method is a helper function that automatically moves the broom to a "lift" height while finding a new position, and then down to a "sweep" height when starting the sweep motion. This allows us to quickly call the function on each detected transform and get the motion we want.

Because the IK policy is deterministic and reliable, the main source of error becomes noise in perception. In our current implementation, if IK fails, we just ignore the object to sweep and move on. While our finetuned YOLO model usually finds the transforms, sometimes partially occluded objects are either not present or misidentified. In the latter case, the predicted transform may be unreachable, leading to an IK failure.

One way this could be addressed but didn't have time to implement is to sample points near the IK destination and run IK on those. Since precise broom placement isn't very important for an

effective sweep, IK destinations in the general area of the target should be sufficient. While this is a small optimization, it could help prevent scenarios where all objects except one or two are in the goal.

4 Final Pipeline Evaluation

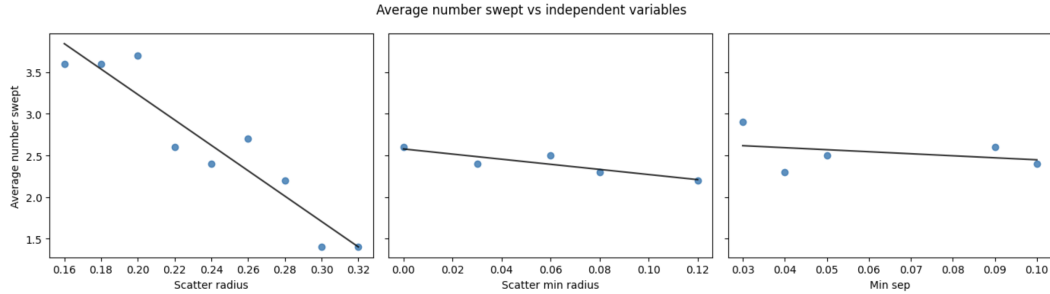


Figure 2: Success Rate of Sweeping over Varied Tests

Unfortunately, the results show the system does very little to solve the problem. Each point on the above graphs was an average over 10 trials for the randomly scattered objects. The *scatter min radius* is the area near the goal in which objects won't be spawned. One thing to note is that, even as the scatter min radius increases, the performance doesn't go down as much as it does for increasing outer radius. This means that even when most of the objects are positioned outside the goal (for 0.08m+), the arm must be getting at least one or two in.

A possible point of failure that explains the poor performance is the use of only one overhead camera for YOLOv8 training. With just one camera, we limited the number of angles that we could cover of each object to just the possible views from a top-down perspective. Once an object was swept to flip over on its side or upside-down, it is very likely the object detection would stop working because YOLOv8 was shown a new angle of the object never seen before. Thus, the transforms would no longer work and the IK would fail to sweep properly from the object to the goal.

Overall, since the sweeping process is deterministic given the transforms, the main source of error lies in perception. With a better tuned YOLO and an extra camera, we would expect to see better performance. As you can see in our demo, the arm does make reasonable sweeping actions, which suggests that when our learned approach failed, the classical approach had at least some merit.

5 References

References

- [1] Et al. Ebrahim H. Sumiea. *Deep deterministic policy gradient algorithm: A systematic review*. ScienceDirect. Accessed: 7 May 2026. URL: <https://www.sciencedirect.com/science/article/pii/S2405844024067288?via%3Dihub>.
- [2] OpenAI. *Deep Deterministic Policy Gradient*. Accessed: 7 May 2026. URL: <https://spinningup.openai.com/en/latest/algorithms/ddpg.html>.
- [3] Amit Yadav. *Reward Function in Reinforcement Learning*. Medium. Accessed: 6 May 2026. URL: <https://medium.com/biased-algorithms/reward-function-in-reinforcement-learning-c9ee04cabe7d>.